Coding Challenge: Product Data Pipeline (Summer Intern 2026)

Context

Cartsy is a shopping agent that converts media content - videos, posts, articles - into shoppable experiences. When a user sees a product in a piece of content, Cartsy automatically tags it, matches it to real purchasable items, and generates shopping links. For this to work well, we need a product database that is fast, accurate, and comprehensive: if a product exists and someone features it in content, we need to already know about it, have clean metadata, and be able to match it confidently even when the reference is informal or incomplete (think "that viral Stanley cup" rather than "Stanley Quencher H2.0 FlowState Tumbler 40oz"). Today we rely on external scrapers and third-party data partners to collect and enrich product information. The quality of our tagging, matching, and link generation is only as good as the data underneath it - duplicates create noise, missing products mean missed conversions, and messy metadata leads to bad matches. Building a reliable product data pipeline is foundational to everything Cartsy does.
Your challenge is to build a small but functional version of this pipeline.

The Challenge

Build a system that ingests product data from multiple sources, normalizes it into a unified schema, deduplicates records across sources, and provides a way to query the results.

Input

We've included **CSV exports from our actual product database.** It includes raw data from different sources, before any enrichment or deduplication on our end. The data is messy in the ways real data is messy: inconsistent field naming, formatting differences across sources, overlapping products, missing values, and varying levels of detail. This isn't contrived - it's what we deal with every day.

What to Build

1. **Ingestion layer:** Parse and load the provided CSVs, handling their structural differences.
2. **Normalization:** Map each source into a common product schema you define. Your schema should be thoughtful: what fields matter for a product database? How do you handle fields that exist in one source but not another?
3. **Deduplication engine:** The core of the challenge. Identify and merge products that appear in multiple sources. You should handle:
    - Exact matches (easy)
    - Near-matches: same product, different naming (e.g., "Sony WH-1000XM5 Wireless Headphones" vs "WH1000XM5 - Sony")
    - Conflicting information between sources (e.g., different prices, descriptions, or categories)
    - Explain and implement a strategy for confidence scoring, “how certain are you that two records are the same product?”
4. **Query interface (optional):** Expose the deduplicated product database through *one* of the following (your choice):
    - A REST API with basic search/filter capabilities
    - A simple web UI that lets someone browse and search products
    - A CLI tool with query commands
5. **Observability:** Basic logging or reporting that answers: How many raw records came in? How many duplicates were found? What's the confidence distribution? Which merges are you least sure about?

Bonus (not required, but impressive)

- **Build a scraper** that pulls real product data from a public source (an open API, marketplace, or catalog page) and feeds it into your pipeline alongside the provided CSVs. This is close to what you'd actually be doing on the job. We want to see how you handle source discovery, extraction, error handling, and integrating a live feed into an existing pipeline.

- Build a **review UI** where a human can inspect low-confidence merges and confirm/reject them.
- Implement a strategy that uses **embeddings or semantic similarity** for deduplication (not just string matching).
- Add **data quality checks, f**lagging records that are likely garbage, incomplete, or suspicious.

Deliverables

Submit a **GitHub repository** (or zip file) containing:

1. **Working code.** It should run. We will clone it, follow your setup instructions, and test it.
2. **README** with:
   - Setup and run instructions (assume we're on macOS or Linux)
   - Architecture overview: what does each component do and why?
   - Your deduplication strategy: how do you decide two products are the same? What tradeoffs did you make?
   - What you'd improve with more time (be specific; "better deduplication" is vague; "I'd add TF-IDF on product descriptions to catch semantic duplicates" is useful)
3. **Sample output.** Include a snapshot of your deduplicated database or a summary report so we can see results even before running the code.

Evaluation Criteria

We're looking at five things, roughly in order of importance:

What	Why it matters
Does it work?	We'll run it. A working system with a simple approach beats an elegant architecture that crashes.
Deduplication quality	This is the hard part. We want to see you reason about fuzzy matching, handle edge cases, and make thoughtful tradeoffs — not

What	Why it matters
	just exact-match on product name.
Code quality and structure	Is it readable? Organized? Would we be comfortable building on top of this code?
Technical decisions	Did you choose the right tools and approaches? Can you explain *why*?
Communication	Your README and code comments tell us how you think. Clear writing about tradeoffs matters as much as the code.

What we explicitly don't care about:

- Choice of language or framework (use what you're fastest with, though Typescript/Node.js is strongly preferred)
- Whether the UI looks polished (functionality over aesthetics on this part of the stack)
- Following any particular design pattern dogmatically
- Writing tests for everything (a few meaningful tests > 100% coverage on trivial things)

Rules

- **Timeline:** 3–4 days from when you receive this.
- **AI tools:** You may (and should) use AI coding assistants, copilots, or LLMs freely. We use them too. But you must be able to explain every architectural decision and walk through any section of code in a live conversation.
- **Libraries:** Use whatever you want. No need to reinvent the wheel.
- **Questions:** If something is ambiguous, make a decision, document your reasoning, and move on. That's a signal, not a problem.

What Happens Next

After you submit, we'll schedule a 30–45 minute walkthrough where you'll:

1. Demo the system end to end

2. Explain your deduplication approach and why you chose it
3. Walk us through a section of code we pick
4. Discuss what you'd change if this needed to handle 10x or 100x the data

Good luck. We're excited to see what you build.